Column generation in Python only?

The pure-Python layer against the C++-backed one

Leo Marbehan

November 9, 2026

The question

C++-backed layer
(`rcspp.column_generation`)

- loop, master problem, pricer in C++
- reached from Python through bindings

Pure-Python layer
(`rcspp.python_column_generation`)

- the same features, written in Python
- no bindings of its own

Both call the **same C++ RCSPP search** for pricing and the same LP solvers (HiGHS through `highspy`, Gurobi through `gurobipy`).

Can we keep only the pure-Python layer?

Yes, if it is as fast in practice. Short answer from the measurements: **same results, same speed within measurement noise on realistic runs**.

Same API in both layers

The benchmark drivers switch layers by **the imported module only**:

```
def layer_module(loop):
    if loop == "purepython":
        import rcspp.python_column_generation as cg
    else:
        import rcspp.column_generation as cg
    return cg

cg = layer_module(loop)                        # the only switch
master = cg.MasterProblem(cg.HighspyMasterSolver(model))   # or GurobipyMasterSolver
graph_pricer = cg.GraphPricer(graph, [cg.PricingStage("simple")])
pricer = master.add_pricer(graph_pricer, cg.ConvexityRow())
result = cg.ColumnGeneration(master, params).run()
result.status, result.objective, result.statistics.generated_columns
```

One difference: the pure layer's options are immutable, set in the constructor (`ColumnGenerationParams(initialization=...)`) instead of by assignment.

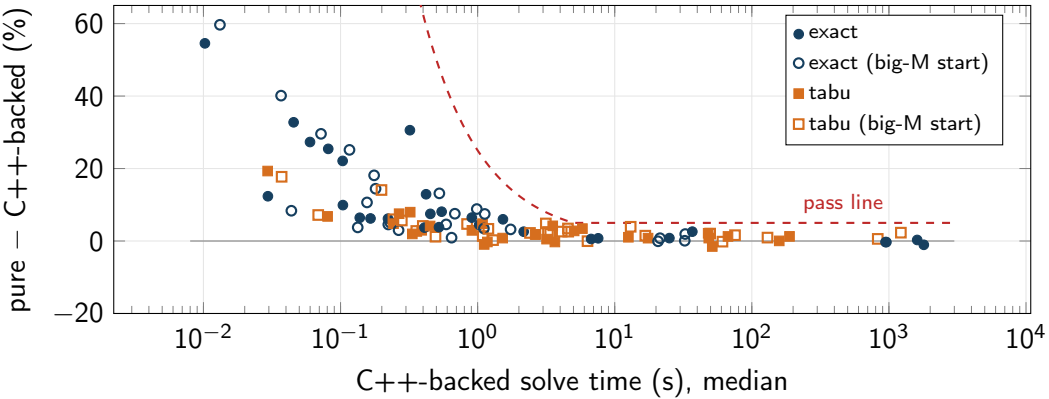
What was run, and correctness

- **108 cases**: 14 Solomon instances (12 to 100 customers) $\times$ {HiGHS, Gurobi} $\times$ {exact pricing, tabu then exact} $\times$ {single-customer routes, big-M start}
- Both layers back to back in each repeat, on one desktop machine
- Repeats by run length: 5 below a few seconds, 3 up to a minute, 1–2 for the longest
- **1,204 runs, 0 failed**

$\Rightarrow$ **Same LP value, same iterations, same columns in every case**: the two layers take exactly the same path; only the time can differ

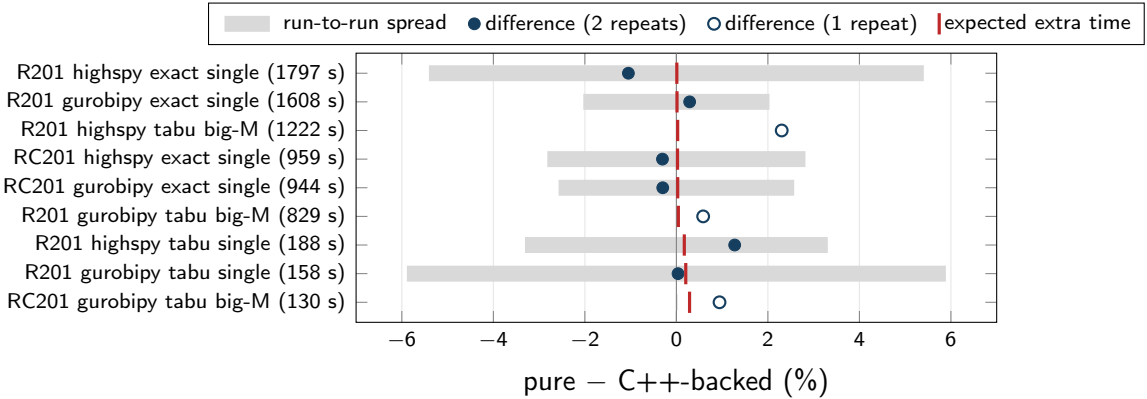
Not run: exact pricing from the big-M start on the two longest instances (about 12 h; pricing takes over 99 % of the time there, so the layers cannot differ by more than the noise).

Extra time of the pure layer, against run length



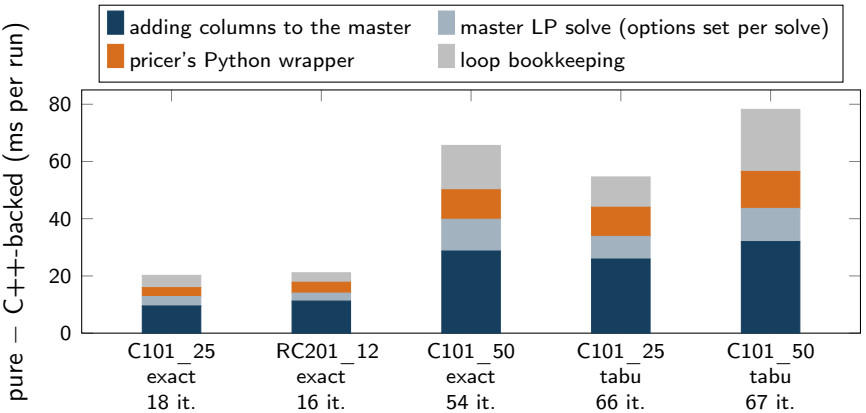
Rule set before the runs: a case passes when the pure layer is at most 5 % slower, or at most 0.25 s slower. **108 / 108 pass** (also with 0.1 s). Below 1 s: up to +60 %, but at most +0.1 s.

Long runs: the difference is smaller than the noise



The 9 cases over 100 s. Grey: how much two repeats of *the same layer* differ (fastest to slowest), 2–6 %. Red: what the pure layer's extra cost per iteration (at most 5 ms) adds up to: under 0.3 %. With 2 repeats, every difference sits inside its case's noise; with 1 repeat (+0.6 to +2.3 %), inside the noise seen on the same instance. The pure layer is faster in 3 of 9. One RC201 re-run even gave it −16 %: another program was busy during the C++-backed run.

Where the extra milliseconds go



About 1–2 ms per iteration (up to 5 ms on the master-heavy C201), plus 2–5 ms once, spread over four places. The pricing search itself is the same C++ call in both layers.

Proposal

1. **Accept the pass rule** as set before the runs (5 %, or 0.25 s for short runs)
2. **Keep only the pure-Python layer**: speed is no longer an argument for the C++ one
   - gain: one implementation; no bindings for the loop; fewer builds, less code; users extend pricers and master solvers in Python
   - cost: **C++ users lose column generation**; we drop the earlier goal "usable from C++ and Python, loop written once in C++". If C++ users matter, that is the reason to keep it
   - work: remove the C++ layer and its bindings, rename the pure package to `column_generation`; rework or close the open C++ pull requests
3. **No C++ helper** to speed up adding columns: it would save at most 0.2 % (C201)

Optional, later: two cheap Python speed-ups (fewer checks when adding columns; not resetting the HiGHS options on every solve).

Caveats

- The longest runs have only 1–2 repeats (up to 30 min each)
- The machine was not fully idle the first night (browser, editor at times up to a full core); runs disturbed by other programs are flagged in the raw results
- All runs on one machine; timings from another machine are not comparable
- Short runs (< 1 s) do show the pure layer slower in %, but by at most 0.1 s

Backup: by run length

C++-backed run	cases	range (%)	median (%)	largest (s)	pure faster
0–1 s	52	+0.9 to +59.7	+7.5	+0.098	0
1–10 s	30	-0.9 to +7.5	+2.5	+0.199	4
10–100 s	17	-1.5 to +3.9	+1.0	+1.215	3
over 100 s	9	-1.0 to +2.3	+0.3	+28.145	3

The % shrinks as runs get longer: the extra cost is per iteration, while the time of an iteration grows with the instance.

Backup: the cases over 30 s

instance	master	pricing	start	C++ (s)	pure (s)	diff (s)	diff	repeats
C201	gurobipy	exact	bigm	32.4	32.4	+0.02	+0.1 %	3
C201	highspy	exact	bigm	32.7	33.3	+0.63	+1.9 %	3
C201	highspy	exact	single	36.8	37.8	+0.95	+2.6 %	3
RC201	highspy	tabu	single	48.0	49.1	+1.03	+2.1 %	2
C201	highspy	tabu	single	48.5	48.9	+0.39	+0.8 %	3
C201	highspy	tabu	bigm	49.7	50.7	+1.02	+2.0 %	3
C201	gurobipy	tabu	single	51.5	50.7	-0.78	-1.5 %	3
C201	gurobipy	tabu	bigm	61.1	61.0	-0.11	-0.2 %	3
RC201	gurobipy	tabu	single	67.0	67.9	+0.84	+1.3 %	2
RC201	highspy	tabu	bigm	75.4	76.6	+1.21	+1.6 %	1
RC201	gurobipy	tabu	bigm	130.1	131.4	+1.23	+0.9 %	1
R201	gurobipy	tabu	single	158.5	158.5	+0.06	+0.0 %	2
R201	highspy	tabu	single	188.3	190.7	+2.40	+1.3 %	2
R201	gurobipy	tabu	bigm	829.1	834.0	+4.85	+0.6 %	1
RC201	gurobipy	exact	single	944.3	941.5	-2.81	-0.3 %	2
RC201	highspy	exact	single	958.6	955.7	-2.91	-0.3 %	2
R201	highspy	tabu	bigm	1222.4	1250.5	+28.14	+2.3 %	1
R201	gurobipy	exact	single	1608.1	1612.8	+4.67	+0.3 %	2
R201	highspy	exact	single	1796.9	1778.1	-18.86	-1.0 %	2

Backup: C201, the master-heavy case

190 iterations, 38,000 columns

	exact	tabu
C++-backed (s)	35.58	46.04
pure (s)	36.52	46.79
difference, total (ms)	+942 (+2.6 %)	+753 (+1.6 %)
master LP solves	+230	+309
pricing	+615	+380
of which adding columns (pure, whole)	342	268
of which the pricer's Python wrapper	108	101
loop bookkeeping	+62	+64

Timed probe over HiGHS, 3 repeats. The one place where the gap is clearly measurable, and still under 3 %.